

Phase 8: Data Management & Deployment

✓ I have managed data and deployment as follows:

✓ Change Sets / VS Code / SFDX: Since I developed directly in a free Developer Edition org, I deployed LWCs using Salesforce Extension. No sandbox-to-production deployment was required.

```
1 <template>
2 <lightning-card title="Find & Track Donations">
3   <lightning-input type="date" label="Start Date" value={startDate}></lightning-input>
4   <lightning-input type="date" label="End Date" value={endDate}></lightning-input>
5   <lightning-button label="Search Donations" variant="brand" onclick={handleSearch}></lightning-button>
6
7   <template if:true={donations}>
8     <lightning-datatable
9       key-field="Id"
10      data={donations}
11      columns={columns}
12      onrowaction={handleRowAction}>
13     </lightning-datatable>
14   </template>
15 </lightning-card>
16 </template>
```

✓ Unmanaged Packages: All objects, LWCs, and triggers are custom unmanaged components created directly in the org and also executed the code in VS.

```
1 import { LightningElement, track } from 'lwc';
2 import getDonations from '@salesforce/apex/DonationService.getDonations';
3
4 export default class DonationSearch extends LightningElement {
5   @track startDate;
6   @track endDate;
7   @track donations;
8   @track columns = [
9     { label: 'Donation Name', fieldName: 'Name' },
10    { label: 'Amount', fieldName: 'Amount__c', type: 'currency' },
11    { label: 'Date', fieldName: 'Donation_Date__c', type: 'date' }
12  ];
13
14  handleSearch() {
15    getDonations({ startDate: this.startDate, endDate: this.endDate })
16      .then(result => {
17        this.donations = result;
18      })
19      .catch(error => {
20        console.error(error);
21      });
22  }
23
24  handleRowAction(event) {
25    // Handle row action, e.g. view donation details
26  }
27
28 }
```

Phase 9: Reporting, Dashboards & Security Review – Donation Tracking System for NGOs

✅ Reporting and Security settings were handled carefully to ensure proper access:

- Dynamic Dashboards: Not implemented since access is controlled via user profiles/roles.
- Sharing Settings & Field-Level Security: Managed using profile and permission-set logic.
 - Donors see only their own donations.
 - NGO Admins see all donations.
 - Volunteers may have read-only access.
- Session Settings & Login IP Ranges: Default org security was sufficient.

Complete Apex Code Example for Donation Management

Apex Class: DonationService.cls

```
public with sharing class DonationService {  
  
    // Inner class to return results to LWC or Aura  
    public class DonationResult {  
        @AuraEnabled public Boolean success;  
        @AuraEnabled public String message;  
        @AuraEnabled public Id donationId;  
  
        public DonationResult(Boolean s, String m, Id d) {  
            success = s;  
            message = m;  
            donationId = d;  
        }  
    }  
}
```

```

    }
}

// Create a donation record with validation
@AuraEnabled
public static DonationResult createDonation(Donation__c donation) {
    // Validate input
    if (donation == null) {
        return new DonationResult(false, 'No donation data provided', null);
    }
    if (donation.Amount__c == null || donation.Amount__c <= 0) {
        return new DonationResult(false, 'Donation amount must be greater
than zero', null);
    }
    if (donation.Donation_Date__c == null) {
        donation.Donation_Date__c = System.today();
    }

    try {
        insert donation;

        return new DonationResult(true, 'Donation created successfully',
donation.Id);
    } catch (Exception e) {
        return new DonationResult(false, 'Error: ' + e.getMessage(), null);
    }
}

```

```

// Fetch donations for reporting/dashboard
@AuraEnabled(cacheable=true)
public static List<Donation__c> getRecentDonations() {
    return [
        SELECT Id, Name, Amount__c, Donation_Datec, Donor__c
        FROM Donation__c
        ORDER BY CreatedDate DESC
        LIMIT 50
    ];
}
}

```

Apex Class: DonationTriggerHandler

```

public with sharing class DonationTriggerHandler {

    public static void validateNoOverlap(List<Donation__c> newDonations) {

        for (Donation__c nd : newDonations) {
            // Check mandatory fields
            if (nd.Donor__c != null && nd.Start_Date__c != null && nd.End_Date__c
            != null) {

                // Query to find any overlapping donations for same donor
                List<Donation__c> conflicts = [
                    SELECT Id
                    FROM Donation__c
                    WHERE Donor__c =:nd.Donor__c

```

```

        AND Status__c != 'Cancelled'
        AND Id != :nd.Id
        AND (:nd.Start_Date__c <= End_Date__c AND :nd.End_Date__c >=
Start_Date__c)
        LIMIT 1
    ];

    // If conflicts found, block save
    if (!conflicts.isEmpty()) {
        nd.addError('This donor already has an active donation during the
selected period.');
```

Apex Class: DonationCallout

```

public with sharing class DonationCallout {

    /**
     * Sends donation data to an external system.
     * @param donationId Id of the Donation__c record
     */
    @future(callout=true)
    public static void registerDonationToExternal(Id donationId) {
```

```
// 1. Fetch Donation Record
```

```
Donation__c d = [  
    SELECT Id, Donor__c, Amount__c, Start_Date__c, End_Date__c  
    FROM Donation__c  
    WHERE Id = :donationId  
    LIMIT 1  
];
```

```
// 2. Fetch Donor Information (external ID)
```

```
Contact donor = [  
    SELECT Id, Name, Donor_ID__c  
    FROM Contact  
    WHERE Id = :d.Donor__c  
    LIMIT 1  
];
```

```
// 3. Prepare HTTP Request
```

```
HttpRequest req = new HttpRequest();  
req.setEndpoint('callout:DonationAPI/donations');  
req.setMethod('POST');  
req.setHeader('Content-Type', 'application/json');
```

```
// 4. Create Payload
```

```
Map<String, Object> payload = new Map<String, Object>{  
    'donationId' => String.valueOf(d.Id),  
    'donorExternalId' => donor.Donor_ID__c,
```

```

        'amount' => d.Amount__c,
        'startDate' => d.Start_Date__c != null ? d.Start_Date__c.format() : null,
        'endDate' => d.End_Date__c != null ? d.End_Date__c.format() : null
    };

    req.setBody(JSON.serialize(payload));

    // 5. Send HTTP Request
    Http http = new Http();
    HttpResponse res = http.send(req);

    System.debug('External System Response: ' + res.getBody());
}
}

```

Apex Class: DonationRegisterQueueable

```

public with sharing class DonationRegisterQueueable
    implements Queueable, Database.AllowsCallouts {

    private Id donationId;

    // Constructor: pass in the Donation__c Id
    public DonationRegisterQueueable(Id donationId) {
        this.donationId = donationId;
    }
}

```

```

// Execute method: calls the external system registration method
public void execute(System.QueueableContext ctx) {
    // Calls our DonationCallout class to send the donation data
    DonationCallout.registerDonationToExternal(donationId);
}
}

```

Apex Class: DonationIssuePublisher

```

public with sharing class DonationIssuePublisher {

    /**
     * Publishes a Donation Issue Event to the Event Bus.
     * @param donationId Id of the Donation__c record
     * @param issueType Type of issue (Payment Failure, Fraud, etc.)
     * @param reportedBy User or system reporting the issue
     */

    public static void publishIssue(Id donationId, String issueType, String
reportedBy) {

        // Create the platform event record
        DonationIssueEvent__e evt = new DonationIssueEvent__e(
            DonationId__c = String.valueOf(donationId),
            IssueType__c = issueType,
            ReportedBy__c = reportedBy
        );

        // Publish the event

```



```

Database.SaveResult sr = EventBus.publish(evt);

// Optional debug log
if (sr.isSuccess()) {
    System.debug('Donation Issue Event published successfully.');
```

```

} else {
    System.debug('Error publishing Donation Issue Event: ' +
sr.getErrors()[0].getMessage());
}
}
}

```

Test Class: DonationServiceTest

```

@IsTest

private class DonationServiceTest {

    @IsTest
    static void testDonationOverlap() {

        // 1. Create a Donor record (Contact or custom Donor object)
        Contact donor = new Contact(
            FirstName = 'Test',
            LastName = 'Donor',
            Email = 'testdonor@example.com'
        );
        insert donor;
    }
}

```

```
// 2. Create an existing donation for this donor
```

```
Donation__c existing = new Donation__c(  
    Donor__c = donor.Id,  
    Start_Date__c = Date.today(),  
    End_Date__c = Date.today().addDays(2),  
    Status__c = 'Active' // not Cancelled  
);  
insert existing;
```

```
// 3. Attempt to create a conflicting donation (overlapping dates)
```

```
Donation__c conflict = new Donation__c (  
    Donor__c = donor.Id,  
    Start_Date__c = Date.today().addDays(1),  
    End_Date__c = Date.today().addDays(3)  
);
```

```
Test.startTest();
```

```
// 4. Call the Donation Service to create donation
```

```
// (Assuming you have a DonationService with a createDonation method)
```

```
DonationService.DonationResult result =  
    DonationService.createDonation(conflict);  
Test.stopTest();
```

```
// 5. Validate it failed due to overlap
```

```
System.assertEquals(false, result.success,  
    'The donation creation should fail due to overlapping dates.');
```

```
}  
}
```

Apex Trigger: DonationTrigger

```
trigger DonationTrigger on Donation__c (before insert, before update) {  
    // Call the handler to validate overlapping donations for the same donor  
    DonationTriggerHandler.validateNoOverlap(trigger.new);  
}
```

HTML: donationSearch.html

```
<template>  
    <lightning-card title="Search & View Donations">  
        <!-- Input for Start Date -->  
        <lightning-input type="date" label="Start Date" value={startDate}  
            onchange={handleStart}></lightning-input>  
  
        <!-- Input for End Date -->  
        <lightning-input type="date" label="End Date" value={endDate}  
            onchange={handleEnd}></lightning-input>  
  
        <!-- Search Button -->  
        <lightning-button label="Search Donations" variant="brand"  
            onclick={searchDonations}></lightning-button>  
  
        <!-- Datatable to display donations -->  
        <template if:true={donations}>
```

```

        <lightning-datatable
            key-field="Id"
            data={donations}
            columns={columns}
            onrowaction={handleRowAction}>
        </lightning-datatable>
    </template>
</lightning-card>
</template>

```

JS: donationSearch.js

```

import { LightningElement, track } from 'lwc';

import searchDonationsApex from
'@salesforce/apex/DonationSearchController.searchDonations';

export default class DonationSearch extends LightningElement {
    @track startDate = '';
    @track endDate = '';
    @track donations = [];

    // Define columns for lightning-datatable
    columns = [
        { label: 'Donation Name', fieldName: 'Name' },
        { label: 'Amount', fieldName: 'Donation_Amount__c', type: 'currency' },
        { label: 'NGO', fieldName: 'NGO__r.Name' },
        { label: 'Phone', fieldName: 'Phone_Number__c' }
    ];
}

```

```
// Handle start date input change
```

```
handleStart(event) {
```

```
    this.startDate = event.target.value;
```

```
}
```

```
// Handle end date input change
```

```
handleEnd(event) {
```

```
    this.endDate = event.target.value;
```

```
}
```

```
// Call Apex to fetch donations
```

```
searchDonations() {
```

```
    // Validate date input
```

```
    if (!this.startDate || !this.endDate) {
```

```
        alert('Please select both Start Date and End Date.');
```

```
        return;
```

```
}
```

```
searchDonationsApex({ startDate: this.startDate, endDate: this.endDate })
```

```
    .then(result => {
```

```
        this.donations = result;
```

```
    })
```

```
    .catch(error => {
```

```
        console.error('Error fetching donations:', error);
```

```
    });
```

```

    }

    // Optional: handle row actions if needed
    handleRowAction(event) {
        const actionName = event.detail.action.name;
        const row = event.detail.row;
        console.log('Row action clicked:', actionName, row);
    }
}

```

JS: donationSearch.js

```

import { LightningElement, track } from 'lwc';
import searchDonationsApex from
 '@salesforce/apex/DonationSearchController.searchDonations';

export default class DonationSearch extends LightningElement {
    @track startDate = '';
    @track endDate = '';
    @track donations = [];

    // Define columns for lightning-datatable
    columns = [
        {label: 'Donation Name', fieldName: 'Name'},
        {label: 'Amount', fieldName: 'Donation_Amount__c', type: 'currency'},
        {label: 'NGO', fieldName: 'NGO__r.Name'},
        {label: 'Phone', fieldName: 'Phone_Number__c'}
    ];
}

```

```
// Handle start date input change
```

```
handleStart(event) {
```

```
    this.startDate = event.target.value;
```

```
}
```

```
// Handle end date input change
```

```
handleEnd(event) {
```

```
    this.endDate = event.target.value;
```

```
}
```

```
// Call Apex to fetch donations
```

```
searchDonations() {
```

```
    // Validate date input
```

```
    if (! this.startDate || !this.endDate) {
```

```
        alert('Please select both Start Date and End Date.');
```

```
        return;
```

```
}
```

```
searchDonationsApex ({ startDate: this.startDate, endDate: this.endDate })
```

```
    .then(result => {
```

```
        this.donations = result;
```

```
    })
```

```
    .catch(error => {
```

```
        console.error('Error fetching donations:', error);
```

```
    });
```

```
}
```

```
// Optional: handle row actions if needed
```

```
handleRowAction(event) {
```

```
    const actionName = event.detail.action.name;
```

```
    const row = event.detail.row;
```

```
    console.log('Row action clicked:', actionName, row);
```

```
}
```

```
}
```